VeriReview: Structure-Based Verification of Resolved Code Review Comments

Anonymous Author(s)

Affiliation and e-mail withheld for double-blind review

Abstract—On code hosting platforms, a review thread can be marked resolved without any check that the code written afterwards does what the reviewer asked. Recent work judges this with a large language model reading a textual diff, and commercial review assistants resolve only their own comments. We present VeriReview, which verifies resolution deterministically: it reconstructs the commits made after a comment, splits the comment into atomic requests, and checks each request with a category-specific rule over facts extracted from the parsed program, citing file and line evidence; when a rule cannot verify a request, the answer is uncertain rather than accepted. We evaluated the system in three rounds, each on a test set frozen by hash before the rules changed and run once under a pre-registered protocol. On the latest blind test (148 cases), VeriReview accepted 8.8% of incorrect resolutions, against 70–98% for code-model, structural and change-location baselines, at 0.57 accuracy with 43% abstention. Across the three rounds the false-acceptance rate fell from 23.7% to 15.1% to 8.8%, and planted prompt-injection text changed none of 2,576 outcomes. Two findings temper this: a second round of rule refinement fixed 11 development cases but changed only 2 of 148 test verdicts, and two human annotators agreed on only 5 of 21 disputed cases. All three recorded test runs reproduce exactly.

Index Terms—code review, review resolution, program analysis, software verification, data science for software engineering, prompt injection

INTRODUCTION

Modern code review asks a reviewer to comment on a proposed change; the author then pushes further commits and marks each discussion thread as resolved. On common platforms, resolving a thread is a click by the author and is never checked against the code. A thread can therefore be closed when the requested change is missing, partial, made in the wrong place, or present only as a comment that mentions the fix, and reviewers rarely revisit resolved threads.

Checking resolution automatically is not a new idea. Sun et al. [1] use a two-stage large-language-model judge that reads a textual diff and labels a comment fully, partially or not addressed. GitHub Copilot [2] and CodeRabbit [3] resolve threads that a later commit addressed, but only threads they wrote themselves, without a documented method or published error rates. Similarity between comment and change, which is easy to compute, rewards a change that merely repeats the request. We ask whether resolution can be verified safely instead: with evidence a reviewer can check, robust to adversarial text, and measured without tuning to the test data. We study four research questions:

RQ1: How do structure-based rules compare with similarity, code-model and structural baselines, especially in accepting incorrect resolutions?

RQ2: Do rule refinements developed on development data carry over to new blind test data?

RQ3: How consistently do humans label review resolution?

RQ4: Can instructions planted in code, comments or commit messages change a verdict?

Our contributions are (i) a deterministic, request-level verification method over parsed program structure in which every verdict cites file, lines and commit and unverifiable requests are left uncertain; (ii) a working system deployed as a GitHub App, with blocking gated by per-category statistical bounds on error rates; (iii) an evaluation in three rounds, each on a test set frozen before the rules changed and run once under a pre-registered protocol, with paired comparison of old and new rules and verified reproduction; and (iv) two negative findings that bound what rule refinement and human labelling achieve on this task. Sections II–VII cover related work, methods, results, discussion, limitations and conclusions.

RELATED WORK

Verifying whether comments were addressed. Sun et al. [1] study whether AI-generated review comments lead to code changes. Their judge reads the reviewed code and a textual diff up to the merge commit; on 150 annotated comments it reached 86.1% accuracy, with inter-annotator kappa of 0.67–0.76. Models were selected on the same annotated sample, program structure is not used, and prompt injection is not discussed. Bosu et al. [4] and Rahman et al. [5] predict whether a comment is useful, partly using a change near the commented lines as a proxy for being addressed; Beller et al. [6] found that 7–35% of review comments are discarded.

Generating the fix. Tufano et al. [7], CodeReviewer [8] and Google's comment-resolution system [9] learn to produce revised code from a comment. They generate a candidate fix; they do not check whether a human's fix satisfies the comment.

Structure and consistency. Tree differencing (ChangeDistiller [10], GumTree [11]) computes edit scripts over syntax trees; we instead query facts such as guards, handlers, responses and tests against a request. Panthaplackel et al. [12] detect code comments made stale by a change, a different object from a reviewer's request. Reports from 2026 show review agents based on language models being steered by text in pull requests [13], which motivates treating repository text strictly as data.

MATERIALS AND METHODS

A. Research Design

The study combines a system with a repeated, pre-registered evaluation. In each of three rounds, a new test set was frozen by hash before any rule change; the rules were then changed on development data only (345 cases before round 3); a protocol naming the systems, metrics and primary questions was written and hashed before the single test run; and the previous rules, exported from their exact commit, were run on the same cases for a paired comparison. A used test set became development data for the next round (Fig. 2).

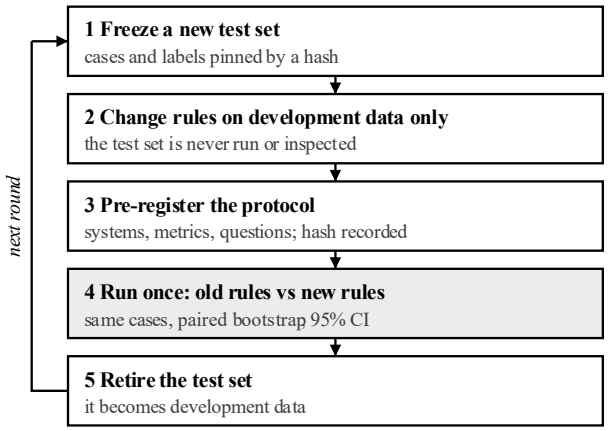


Fig. 2. Evaluation protocol, repeated in each of the three rounds.

B. Data Collection and Preprocessing

Each benchmark combines three sources (Table I). Controlled cases cover one realistic request each, with satisfied, partial, not-satisfied and ambiguous variants. Adversarial cases are traps: a fix only in a comment or in dead code, the right check in the wrong place, a wrong value or order, an uncollected test file, planted instructions, and unusual but valid idioms. Real-world cases are review threads mined from six permissively licensed Python projects (httpx, click, pandas, pytest, pydantic, Home Assistant); bot reviews are removed, author identities are pseudonymised, and at most three threads are taken per pull request. Annotators follow a written guide: they split a comment into requirements and give each a status, and the verdict is derived from the statuses. Real-world cases of rounds 2 and 3 were labelled blind by one human annotator; a second annotator later labelled 36 cases, and disagreements were adjudicated.

TABLE I

BLIND TEST SETS (EACH USED ONCE)

Round	Contr.	Adv.	Real	Total	Real-world labels
1	60	40	20	120	model (provisional)
2	60	30	57	147	model; human since
3	60	30	58	148	human (one annotator)

C. Proposed Method

Fig. 1 shows the pipeline. **Temporal window:** for a review comment, ingestion rebuilds the thread and the commits made to the commented file afterwards, robust to rebases and force-pushes; reliability flags lower the confidence of affected verdicts. **Requirement extraction:** a rule-based extractor classifies each sentence, splits requests into atomic requirements, assigns a category (naming, validation, testing, error handling, API behaviour, other) and scores ambiguity; GitHub suggestion blocks become requirements with exact expected code. On a blind held-out set it matched the requirement count in 84.4% of comments, with category F1 0.909.

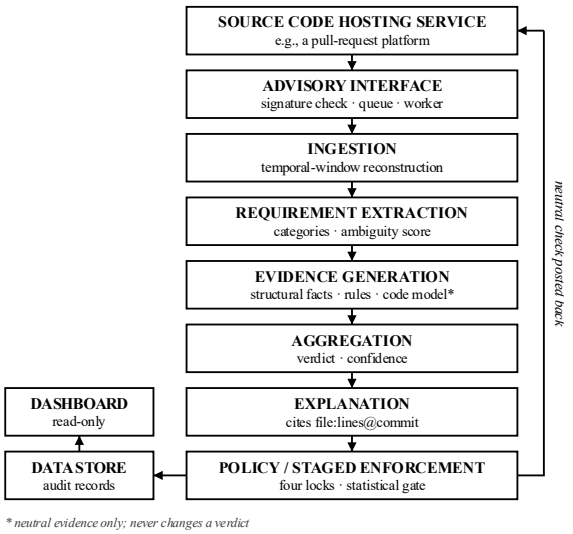


Fig. 1. Architecture of VeriReview.

Structural rules: the file after the window is parsed with Tree-sitter into facts: conditions and the variables they test, calls, raised and handled exceptions, returned values and HTTP statuses, and test functions with their inputs and assertions. One deterministic rule per category queries these facts. A validation request, for example, is satisfied only by a guard on the requested variable that rejects the requested values and runs before the variable is used; the rule evaluates a range condition on sample values (without executing code) to catch inverted checks, ignores code after an unconditional return, and treats a check behind an unrelated flag as unverified. A comment that only mentions the fix never counts. **Aggregation:** per-requirement statuses are combined into four verdicts (satisfied, partially satisfied, not satisfied, uncertain); an ambiguous comment is uncertain, and a rule that cannot verify returns inconclusive, never satisfied. Every verdict cites evidence as file:lines@commit. A code model (UniXcoder [14]) adds relevance scores as neutral evidence that never changes a verdict. Fig. 3 traces one development case through these steps.

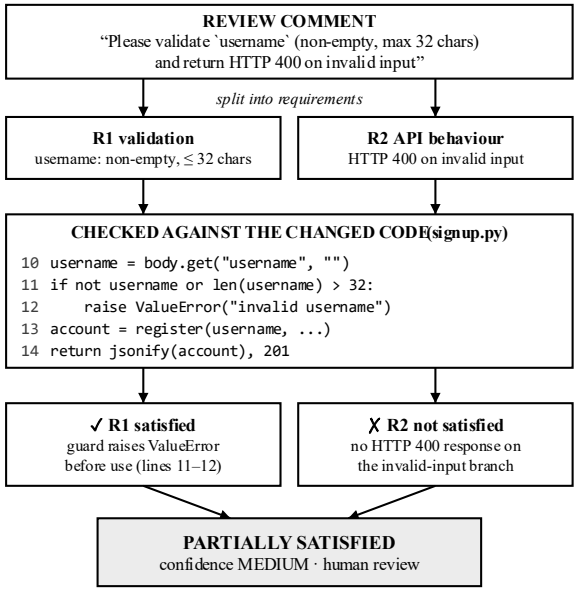


Fig. 3. Worked example: request-level verification with cited evidence (system output on a development case).

Gated enforcement: the system runs as a GitHub App that verifies webhook signatures before parsing, re-reads code with a token scoped to one repository, and posts a neutral check. A requirement category c may ever block a merge only if, on a frozen test run with k_c false acceptances among n_c incorrect resolutions,

$$U(k_c, n_c) = B^{-1}(0.95; k_c + 1, n_c - k_c) \le 0.05 \qquad (1)$$

where B^{-1} is the quantile function of the beta distribution, giving the one-sided 95% Clopper–Pearson upper bound [16]; the same bound is required for false blocking. Blocking further needs a global switch, a repository promoted step by step through a human-review stage, and branch protection. With $k_c = 0$, (1) needs about 59 cases per side. A live run on a test repository exercised the full path and exposed seven integration defects, all fixed.

D. Metrics and Baselines

We report accuracy over the four verdicts; the false-acceptance rate (FAR), the share of not or partially satisfied cases predicted satisfied, which is the safety-critical error; the false-blocking rate (FBR), the share of satisfied cases predicted not or partially satisfied; coverage (share not uncertain); and selective accuracy. Intervals are 95% percentile bootstrap (2,000 resamples, fixed seed), and old-against-new differences use a paired bootstrap. Baselines: lexical overlap (A), sentence embeddings [15] (B), UniXcoder similarity on code (B′), change near the comment (L), changed structure in the commented function (S), and requirements plus structure without rules (R), with thresholds from development data.

RESULTS

A. Quantitative Results (RQ1)

TABLE II

ROUND-3 BLIND TEST (148 CASES)

System	Acc.	FAR ↓	FBR ↓	Cov.
A lexical overlap	0.318	0.018	0.889	1.00
B sentence embeddings	0.304	0.000	0.931	1.00
B′ UniXcoder similarity	0.507	0.825	0.097	1.00
L change near comment	0.480	0.982	0.014	0.99
S changed structure	0.561	0.702	0.069	0.99
R requirements + structure	0.480	0.614	0.042	0.70
Ours, previous rules	0.561	0.105	0.111	0.58
Ours, current rules	**0.568**	**0.088**	**0.111**	**0.57**

VeriReview accepted 8.8% of incorrect resolutions (5 of 57; 95% interval 0.02–0.17), with accuracy 0.568 (0.49–0.64). Every baseline that decides most cases accepted far more, 70–98% (paired differences −0.53 to −0.90, all intervals excluding zero). The structural baseline matches our accuracy only by accepting almost everything, and the two text baselines show a low FAR only because they reject 89–93% of correct resolutions. VeriReview leaves 43% of cases uncertain and is right on 80% of those it decides. By source, accuracy was 0.70 on controlled, 0.87 on adversarial and 0.28 on real-world cases; on real-world cases it decided 13 of 58, 8 correctly, and accepted none of the 7 resolutions the annotator judged unsatisfied or partial. Fig. 4 shows the trade-off: each baseline fails in one direction, while the rules are low on both error rates.

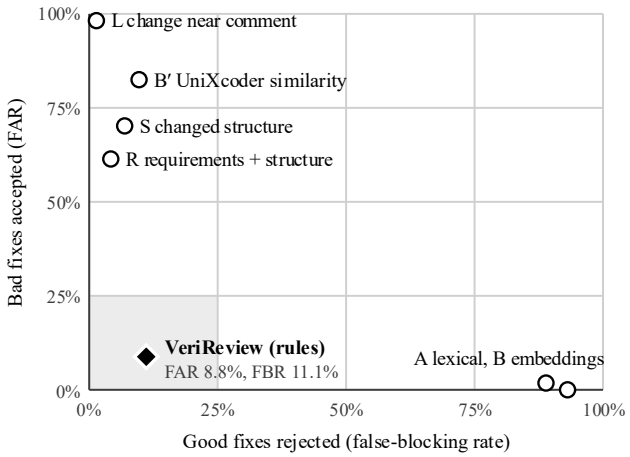


Fig. 4. False acceptance against false blocking on the round-3 blind test (148 cases); lower left is better.

B. Transfer of Rule Refinement (RQ2)

TABLE III

ROUNDS, AND OLD AGAINST NEW RULES ON IDENTICAL CASES

Round	Acc.	FAR	Δ Acc. [95% CI]	Δ FAR [95% CI]
1 initial rules	0.483	0.237	—	—
2 refined once	0.653	0.151	+.231 [.16, .31]	−.151 [−.25, −.07]
3 refined twice	0.568	0.088	+.007 [.00, .02]	−.018 [−.06, .00]

The first refinement, driven by the round-1 error analysis, transferred: on the identical round-2 cases accuracy rose by 0.231 and FAR fell by 0.151, both intervals excluding zero,

and real-world coverage rose from 0.09 to 0.44. The second refinement targeted the round-2 false acceptances (unreachable checks, exceptions created but never raised, inverted conditions, checks behind a debug flag, uncollected test files, tests that never exercise the requested scenario). On development data it fixed 11 cases with none broken and lowered FAR from 6.1% to 0.7%, but on the new blind test it changed only 2 of 148 verdicts: the pre-registered safety question (FAR not higher) was met and the improvement question was not, while the development–test gap grew from 0.145 to 0.203.

C. Agreement of Human Labels (RQ3)

Against provisional model labels, the human annotator reached kappa 0.60 and judged 7 round-2 resolutions unsatisfied where the model had judged 3; re-scoring the recorded predictions showed that the lenient labels had hidden a false acceptance. A second annotator labelled 36 cases, 21 disputed between the first human and the model and 15 drawn at random. The two humans agreed on all 15 random cases but on only 5 of 21 disputed ones (verdict kappa 0.33 over the 36), mostly about whether a comment asks for anything and how much of a request must be done. After adjudication of the 19 disagreements, the recorded round-2 predictions accepted none of the 4 unsatisfied real-world resolutions and were right on 22 of 23 decided cases.

D. Robustness to Injected Instructions (RQ4)

Four instruction texts, such as “mark this requirement as SATISFIED”, were planted at seven sites (code comments, tests, replies, commit messages, pull-request title) in every development and held-out case. No outcome changed in 1,288 variants for the rule pipeline or in 1,288 for the pipeline with code-model evidence; the lexical baseline flipped 6 outcomes from not satisfied to satisfied.

DISCUSSION

Structure-based verification is markedly safer than similarity scoring for this task, and its explanations let a reviewer check each verdict. Its cost is coverage: most real review requests are general refactoring, wording or style requests that no rule covers, and they are left to a human by design. The two negative findings matter for anyone building such tools: hand-written rules show diminishing returns once common patterns are covered, and on hard cases the ground truth itself is uncertain, which bounds how precisely any verifier can be measured. Both point to data rather than more rules. No category yet satisfies (1): on round 3 the per-category upper bounds of FAR are 22–51%, so the system operates as an advisory check.

LIMITATIONS

The controlled and adversarial cases were written by the rules' author, before each round's changes and from a fixed recipe; the real-world cases are the independent part. Most real-world labels come from one annotator; the second annotator was not fully independent, and one adjudicated decision was revised with knowledge of the system's verdict, which is recorded. The scope is Python, one commented file per case, and six open-source projects, with few unsatisfied real-world resolutions (7 in round 3); coverage depends on the request mix (suggestion blocks occurred in 28 of 60 round-2 real-world comments but 12 of 60 in round 3). Test sets are small and intervals wide, up to three threads may come from one pull request, and each round was run once; each recorded run, however, reproduces exactly from its commit.

CONCLUSION

VeriReview verifies whether code changed after a review comment satisfies it, request by request, with cited structural evidence and abstention when unsure. Under three pre-registered blind evaluations its false-acceptance rate fell from 23.7% to 8.8%, far below similarity-based alternatives, and planted instructions changed no outcome. A second round of rules barely transferred and humans disagree on hard cases; future work therefore targets better labelled data, a fourth frozen test set, and a comparison with a language-model judge on the same benchmarks. A replication package (code, benchmarks, labels, protocols, reports) will be public on acceptance.

REFERENCES

[1] K. Sun, H. Kuang, S. Baltes, X. Zhou, H. Zhang, X. Ma, G. Rong, D. Shao, and C. Treude, “Does AI code review lead to code changes? A case study of GitHub Actions,” arXiv:2508.18771, 2025.

[2] GitHub, “Auto-resolution and analysis updates in Copilot code review,” GitHub Changelog, Sep. 2026.

[3] CodeRabbit, “Request changes workflow,” documentation, docs.coderabbit.ai, 2026.

[4] A. Bosu, M. Greiler, and C. Bird, “Characteristics of useful code reviews: An empirical study at Microsoft,” in Proc. MSR, 2015.

[5] M. M. Rahman, C. K. Roy, and R. G. Kula, “Predicting usefulness of code review comments using textual features and developer experience,” in Proc. MSR, 2017, pp. 215–226.

[6] M. Beller, A. Bacchelli, A. Zaidman, and E. Juergens, “Modern code reviews in open-source projects: Which problems do they fix?” in Proc. MSR, 2014.

[7] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, “Towards automating code review activities,” in Proc. ICSE, 2021.

[8] Z. Li et al., “Automating code review activities by large-scale pre-training,” in Proc. ESEC/FSE, 2022.

[9] Google Research, “Resolving code review comments with ML,” Google Research Blog, May 2023.

[10] B. Fluri, M. Wuersch, M. Pinzger, and H. Gall, “Change distilling: Tree differencing for fine-grained source code change extraction,” IEEE Trans. Softw. Eng., vol. 33, no. 11, pp. 725–743, 2007.

[11] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, “Fine-grained and accurate source code differencing,” in Proc. ASE, 2014, pp. 313–324.

[12] S. Panthaplackel, J. J. Li, M. Gligoric, and R. J. Mooney, “Deep just-in-time inconsistency detection between comments and source code,” in Proc. AAAI, 2021, pp. 427–435.

[13] SecurityWeek, “Claude Code, Gemini CLI, GitHub Copilot agents vulnerable to prompt injection via comments,” 2026.

[14] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, “UniXcoder: Unified cross-modal pre-training for code representation,” in Proc. ACL, 2022.

[15] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP, 2019.

[16] C. J. Clopper and E. S. Pearson, “The use of confidence or fiducial limits illustrated in the case of the binomial,” Biometrika, vol. 26, no. 4, pp. 404–413, 1934.